

Peter Naur's view of programming (hiringengineersbook.com)

292 points by cloogshicer on Feb 4, 2021 | hide | past | favorite | 190 comments

QuercusMax on Feb 4, 2021 | next [-]

The point about theory-building requiring (or at least being accelerated by) interpersonal communication / teaching rings very true.

In the middle of last year, my team went through a major re-org, and I'm now working with a whole bunch of new teammates. My project didn't get cancelled in the reorg; in fact, it's actually come to more prominence due to synergy in the projects we work on. Essentially my new teammates are working on applications which have 2-4 years of catching up to reach the same level of maturity as my existing work.

It's taken me the better part of a year to communicate the whys, whats, and hows of the systems we've built. I've given multiple talks, explained in 1:1s, written docs, and it's just taken a really long time to get across these ideas. All-remote work has definitely made a big (negative) impact in the velocity that ideas can be communicated.

inadequatespace on Feb 15, 2021 | parent | next [-]

> All-remote work has definitely made a big (negative) impact in the velocity that ideas can be communicated.

This, along with the fact that employment half-life is long relatively to COVID (2 years average tenure), convinces me that this "remote permanent" hype is going to grind to a screeching halt in another year or so.

Yes, I'm putting my money where my mouth is (in SF Bay real estate, in this case).

mssundaram on Feb 4, 2021 | parent | prev | next [-]

> *All-remote work has definitely made a big (negative) impact in the velocity that ideas can be communicated.*

It doesn't have to be that way, but if that's new to you and your team, I can understand that happening

dasyatidprime on Feb 5, 2021 | root | parent | next [-]

I dunno, "it doesn't have to be that way" is very contextual. I think the *peak* group coherence and idea integration available in physical proximity is probably considerably above the peak available with current-day telecommunications (and current-day social/psychological technologies/practices surrounding it). But of course most groups won't be able to reach either peak, and the peaks for particular individuals or groups may be reversed from that, or there may be other factors that reverse them. (For instance, being able to more legibly put effort into communication practices with the *excuse* of "we need to relearn to work together because remote", even if doing the same thing while in proximity would have had even better effects—and not necessarily because of external pressure, since the same emotive mechanic can operate *within* the group.)

sriku on Feb 5, 2021 | root | parent | prev | next [-]

One of the complaints about remote work at my company that I keep hearing (and also feel myself) is that our staff miss having "random" conversations. I quote random, because it isn't about the literal randomness, but kind of closer to "unstructured and unintended" conversations. We run an ideation-to-prototype event at the company and currently the teams are struggling with ideation that used to happen in such a "random" manner rather easily in person.

striking on Feb 5, 2021 | root | parent | next [-]

At my workplace, we have a few blocks of time scattered around the schedule for teammates to just jump in and code with other people. No agenda, no one is required to join. But a few always do, because it's so pleasant.

johbjo on Feb 5, 2021 | parent | prev | next [-]

It sounds like the new teammates are too comfortable if their current learning rate is 2-4 years. That number could also be an overestimation, which reinforces the relative rank in the team.

This points to another "field of tension": when the value of the company is in the minds of the programmers (not in the code), how should an organization handle rank such that all team members become stakeholders in the most effective progress?

I suspect more like academic rank rather than "team lead" or "lead dev".

QuercusMax on Feb 6, 2021 | root | parent | next [-]

My field, medical device software, has very long product cycles. It's not that my teammates are taking a long time to learn, it just takes time for a project to reach maturity.

Additionally, we've all been learning to collaborate in a new mode, with immense distractions. Prior to our reorg, we'd never actually worked together before. And we all had our own deadlines we were trying to meet. The last 11 months have been pretty awful, ya know?

rorykoehler on Feb 4, 2021 | parent | prev | next [-]

Remote work should help you in this case. Writing and reading are scalable. 1:1's are not for explaining how stuff works and why. If you don't write this all down then you are introducing risk and key person dependency.

closeparen on Feb 4, 2021 | root | parent | next [-]

It's not a one-way transfer of information.

- The student has questions that it wouldn't have occurred to the teacher to answer.
- The student gets confused and asks for things to be rephrased or reframed.
- The student has questions that don't currently have static answers, but get computed by the teacher in real time, according to an intuition and assimilation of the facts that the student is only beginning to develop.
- The teacher's answers are up to date, whether or not he has actually exercised the diligence to maintain the textbook.
- The teacher fields the questions that the specific students ramping on the project actually have, rather than trying to anticipate all possible questions of all hypothetical students (and still failing).

There is a reason we have college and not just reading lists. And those are subjects where the economics support massive investments in discovering the best / most broadly useful ways to present the ideas. The average software project isn't that.

The commoditized software factory is an MBA fantasy. The expertise held by "key persons" is a software team's greatest asset.

rorykoehler on Feb 5, 2021 | root | parent | next [-]

I learned f all in college and everything through reading and watching back video tutorials/lectures. YMMV.

simiones on Feb 5, 2021 | root | parent | next [-]

Then you had poor teachers or a poor structure that didn't give you access to teachers in the right way.

While documentation has both advantages and disadvantages compared to in-person training, video tutorials are strictly worse than an interactive lecture.

rorykoehler on Feb 5, 2021 | root | parent | next [-]

I'm an autodidact. No teacher can provide the density and velocity of information and knowledge the Internet, science papers and books can provide. I can't watch a lecture at 2x speed if its live. I can't cherry pick. I have to be at a certain place at a certain time when I might not be in the right headspace. Then add on top that's it's basically a crapshoot whether or not you get along with your teacher or not. I don't regret going to college but I didn't gain much academic knowledge there either.

jimmyvalmer on Feb 5, 2021 | root | parent | next [-]

100%. As an undergrad in the 90s I came to this conclusion as well. Then when YouTube, Khan Academy, and MOOCs hit critical mass in the 2010s, I could see the beginning of the end. COVID-19 has *greatly* accelerated the demise of in-person learning, and the fact that colleges continue to charge the same tuition is clear evidence that what they're selling isn't education, it's credentialling.

thomasahle on Feb 5, 2021 | root | parent | prev | next [-]

> I can't watch a lecture at 2x speed if its live. I can't cherry pick.

If you have a 1:1 teacher you can just tell them what you already know and they can jump to the parts you don't. Much more efficient than skipping through a video for interesting snippets.

rorykoehler on Feb 5, 2021 | root | parent | next [-]

I think 1:1 or even one to many teaching is amazing but more for the cutting edge stuff. It's changing how you think more than what you know.

solatic on Feb 4, 2021 | root | parent | prev | next [-]

Reading and writing scale *in theory*. They require an organization staffed with people who are strong readers and writers. Unfortunately, many people, especially those forced into remote work during the pandemic, are not. They will not read through explicit, clear, but long documentation; they will not respond by writing questions of their own, and will often fail at writing explicit and clear documentation of their own.

You are who you hire.

rorykoehler on Feb 5, 2021 | root | parent | next [-]

....They won't do their job.

[simiones](#) on Feb 5, 2021 | [root](#) | [parent](#) | [next](#) [-]

Their job is not to write docs, it's to produce a product. The docs are a possible tool towards that goal, but there are others, with different trade-offs - such as in-person trainings.

It is fairly obvious that most of the industry believes the opposite from you here - thorough docs are an extremely expensive way of achieving good context, and as such are usually replaced with training sessions, which seem to fit most people better up to some point.

[rorykoehler](#) on Feb 5, 2021 | [root](#) | [parent](#) | [next](#) [-]

You can't do in-person training properly without docs. It's not an either or choice.

[ilaksh](#) on Feb 4, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

I think it should help in a way. But I also believe that people overestimate how well one can learn in depth about a complex problem from reading or any mainly passive activity alone.

I know there is this theory about learning styles but it's largely debunked.

To get a really solid understanding of a topic such as a business domain and it's interaction with a complex process and application, it takes a certain amount of time and contact with real problems. The only time that people can quickly pick up new ideas from reading is when they have mastered all of the underlying concepts of that knowledge. But applications generally have layers of very specific knowledge required to understand the problem and existing solution.

The point of the article I think is that it's usually going to take a significant amount of time for new developers to become familiar, regardless of how good the source or documentation is.

[QuercusMax](#) on Feb 4, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

Unfortunately for me, I started developing really bad RSI around January of last year, and typing has been incredibly painful. In the past I would have discussed things using whiteboards, sitting down side-by-side with code, etc.; these are all very hard to do if typing causes pain.

Interactive sessions are also very helpful, as there are so many assumptions and background material baked into things that often take a long time to unwind. Being able to gauge on-the-fly if the audience understands can save tons of time and confusion.

I've got 17 years of domain experience across 3 companies in the industry; there's no way I'm going to write all that down in my docs.

[hathawsh](#) on Feb 5, 2021 | [root](#) | [parent](#) | [next](#) [-]

It sounds like you need a secretary--someone who can type fast and dress up your words for wider distribution in the company.

[QuercusMax](#) on Feb 6, 2021 | [root](#) | [parent](#) | [next](#) [-]

You're not wrong; I've actually thought about that.

Fortunately my RSI is starting to get better. Therapeutic massage and shoulder rehab exercises have made a big difference (I had a bike accident a few years back, and my shoulders were in pretty bad shape).

I stopped typing on my 2018 MacBook Pro's keyboard (which I think contributed to my problems), and

exclusively use an ErgoDox. I've also been taking Magnesium as well as NSAIDs to reduce inflammation. Still a daily struggle, but I can type posts like this one without too much pain.

sarchertech on Feb 5, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

Dragon works very well for dictation, and a Wacom style tablet works great as a remote substitute for whiteboarding.

There are also voice coding solutions like Talon voice that work well (I use it regularly).

hydroxonium on Feb 5, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

Might be worth trying Dvorak keyboard layout.

Has worked for many, me included.

cloogshicer on Feb 4, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

I think the point that Naur was trying to make is exactly that this process (unfortunately) doesn't scale.

That is my experience at least (even in very well documented projects).

chubot on Feb 4, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

It depends on the situation. Documentation and in-person meetings aren't mutually exclusive. A meeting to go through the docs and update them is often very useful.

Not everything has to be "scalable". There are many parts of the code that only 1 or 3 people will ever work on. In fact in most places I'd say that's most of the code.

rorykoehler on Feb 4, 2021 | [root](#) | [parent](#) | [next](#) [-]

I've never been unhappy when provided with thorough documentation.

majormajor on Feb 4, 2021 | [root](#) | [parent](#) | [next](#) [-]

Do you have any examples of sufficiently thorough documentation?

I've basically never been happy with the documentation I've been provided with, whether for internal company code or 3rd party OSS stuff or 3rd party paid stuff...

rorykoehler on Feb 5, 2021 | [root](#) | [parent](#) | [next](#) [-]

Just do audits. There are ISO standards for practically everything and you should run internal audits (and external ones if you can afford it for important stuff).

lggleSniggle on Feb 4, 2021 | [prev](#) | [next](#) [-]

I am sometimes guilty of reading the comments before reading the article. I often find the reactions more concise, insightful, and valuable to me than the source material.

That said:

If you are reading this comment and haven't read the article, go read the article instead.

This is an inverse scenario: imho most of the comments here are interesting but have only marginal value building from the insight of the article.

CryptoGhost on Feb 4, 2021 | parent | next [-]

The author doesn't make a good case for "The main value of a software company is the mapping of source code and problem space in the developer's heads" being a universal truth. There are some problem domains and practical development and deployment roadblocks that simply make certain source code more valuable than others. Nowadays it seems people are more willing to throw money at open sourcing solutions so competing firms don't have to roll their own but that's not a hard and fast phenomenon.

mjburgess on Feb 5, 2021 | root | parent | next [-]

I think "software company" can probably hold the weight of the argument, if defined well.

ie., a Bank maybe finds most of its value in *that* its code runs, etc.

A software company, perhaps, we could define (1) simply as any company where this is true; or if that's cheating, (2) any company where its competitive advantage derives from its software developing over time.

I think the claim of this article is that (1) and (2) refer, approximately, to the same companies: ie., that software which is changed to adapt to ever changing needs derives that capacity from the "theory of the programmers" and not the code.

IggleSniggle on Feb 5, 2021 | root | parent | prev | next [-]

I agree, but at least for me, this did not diminish the value of the article.

vannevar on Feb 4, 2021 | prev | next [-]

"Worthless" is an exaggeration: any significant body of code running a critical business process would be missed---and expensive to recreate---if it were lost. But I think the article does make a good point about the value to others. Unless you have a dev team whose head is in that code, and a business team built around that software workflow, just having the code won't be of much value to you. The value of software is inextricably linked to the team that has shaped it. So while it isn't worthless, it is certainly of much lower value without that team.

toss1 on Feb 5, 2021 | parent | next [-]

You are certainly correct about the costs of losing access to the source code, e.g., if it were destroyed.

However, it seems the author was discussing a different meaning of "lost", as in loss of control of the source and having it revealed to the outside world. He made the best arguments I've seen that the bulk of it is worthless to anyone else, who would be better off rolling their own from scratch, and provided a good example.

Indeed, if I were working a problem and somehow ethically came into possession of a competitor's code or product (e.g., via a buyout), the only use I'd put it to would be to try to see if they had any unique insights that we could adapt, and maybe some stand-alone chunks to use. And, indeed, that was the fate of a codebase of which I was quite proud when a company I co-founded got bought.

cloogshicer on Feb 4, 2021 | parent | prev | next [-]

Agreed, worthless might have been a bit exaggerated.

reactordev on Feb 4, 2021 | root | parent | next [-]

worthless-to-outsiders is more concise to me that conveys what the author was intending.

ncmncm on Feb 4, 2021 | parent | prev | next [-]

A better description would be "a liability". Every extra line of code is a line that has to be maintained in perpetuity. Each new engineer has to learn it all again.

The saleable product has value. A library you use for multiple products has some value. Everything else is cost.

username90 on Feb 5, 2021 | root | parent | next [-]

Engineers are a liability. You need them to write and fix code that runs your product, but other than that every extra engineer just demands salary and increases your costs.

Therefore I conclude that engineers have no value for a company, since when ignoring all the value they provide they just cost money.

atomicson on Feb 5, 2021 | root | parent | next [-]

It depends on size of the company. Cost is a relative instrument. Develop a big market product, highly demanding, or for a critical mission application is another story for "cost".

bennysonething on Feb 4, 2021 | prev | next [-]

This is probably one the most poignant things I've read about programming. I've recently been through a handover, left on my own at a new job, with a long back log full of fairly useless stories, left to forge relationships with the business. Mostly left depressed and stressed. All the while a manager said "I don't understand why the handover is taking so long, it's all documented". Yeah the documentation didn't really tell me much as I lacked the business context. I'm brand new to this domain too. The article has put into words what I've been feeling for the last year.

musicale on Feb 4, 2021 | prev | next [-]

> Retaining talent is even more important than you might think. It's crazy that in an industry where the main value is tied up so much to individual contributors, people change jobs every two years

For most companies I'm familiar with, switching jobs (usually to a different company) is the most effective (and often the only) path to achieving a significant raise or promotion without moving into management.

plutonorm on Feb 5, 2021 | parent | next [-]

Where does everyone get the impression that you can just move into management? It's certainly not the case in circles I move in. In 15 years I've not seen a single promotion into management. That's in London.

devlopr on Feb 5, 2021 | root | parent | next [-]

I think London England has a class structure that discourages this. In North America the push to team leader happens quite often and middle management common. The glass ceiling is usually cto/ceo/coo.

jagged-chisel on Feb 5, 2021 | root | parent | prev | next [-]

In the States, it's not "just move into management," but an expectation from management that their positions are "above" those who aren't management, and once an employee maxes out salary at their skill (because we certainly won't pay the peons more than management...) they only promotion is management. So go take some classes and work your way up that corporate ladder...

plutonorm on Feb 5, 2021 | root | parent | next [-]

This is the opinion of management in England too. Except here there is no way to break into management without going back to school and even then it won't be easy to shake the "developer" image. I'm currently doing a startup, not because I want to do startups, but because I need to get out of development for the sake of my mental health and starting your own company is the only available avenue I have for achieving the transition.

hallareempt on Feb 4, 2021 | prev | next [-]

My source code is invaluable. It's open, public, free to reuse under the GPL. It's a complete implementation of a painting application used by millions of people. It's a huge amount of knowledge readily accessible. You won't be able to buy something like this for any sum of money, so it's free.

cloogshicer on Feb 4, 2021 | parent | next [-]

Is it really the source code that has value, or your continued expansion and development of said code? Genuine question.

The reason I'm asking, while I was writing the article, I was considering including the youtube-dl fiasco (when the RIAA took it down from GitHub) as an example. People were concerned not because the code was gone (lots of mirrors popped up quickly), but because they were worried that the contributors would stop further development.

I think that's another indicator that the code itself carries little value. However, the problem space you've loaded and mapped to code in your own head, does have lots of value. Of course the fact that you're sharing it with the world in the best format we know so far (code) for free is much appreciated :)

gpm on Feb 4, 2021 | root | parent | next [-]

Youtube-dl is an app chasing a moving operating of trying to interact with other sites APIs... many of which don't really even want to let youtube-dl do that. Most apps aren't in such a difficult space and keep working basically forever.

cloogshicer on Feb 4, 2021 | root | parent | next [-]

> Most apps aren't in such a difficult space and keep working basically forever.

Sadly, this doesn't match my experience. Software that isn't actively maintained always dies, sooner or later.

btilly on Feb 4, 2021 | root | parent | next [-]

Whether this is true depends on what you do.

The last update to TeX (widely used in math and computer science for typesetting) was 12 January 2014.

A lot of payment processing systems are still running on code written in the 1960s and 1970s. Frequently untouched since Y2K.

I have a friend who went to work in the mid 2000s for a company she had worked for in the early 1970s. Out of curiosity she looked up her old programs. They were still running, unchanged. She asked why and was told, "They never broke."

One of the reasons for the survival of FORTRAN is that there are trusted software packages that people rely on which were written decades ago and still run.

There is an active emulator community for people who want to run games that are decades old, unchanged.

No, your old Netscape browser won't work in the modern web. Nor are early mobile apps going to run. But you'd be amazed at how many places you can find old software still happily running today.

Jtsummers on Feb 4, 2021 | root | parent | next [-]

Good modular design is helpful here. Properly segregating responsibilities means that portions of your code base can become "finished", while other portions remain in near constant flux. For an emulator, for example, if you separate the rendering from the hardware emulation portion, you can leave the hardware emulation portion untouched for years at a time while changing just the rendering code to port to new platforms.

The best "old" (30+ or 40+ years) code I worked on did this. The worst, which forced total rewrites, mingled everything together "for performance" but prevented the software from being easily ported to a new OS (Windows 3.1 hasn't been supported for a *long* time) or extended to support new capabilities.

Imm on Feb 5, 2021 | root | parent | prev | next [-]

> The last update to TeX (widely used in math and computer science for typesetting) was 12 January 2014.

But if you want to actually use it you'll install a much more recent distribution - probably texlive from 2020.

btilly on Feb 5, 2021 | root | parent | next [-]

Yes. You'll install a bunch of recent stuff around it.

But the heart of it, TeX, is still the same old version.

Imm on Feb 5, 2021 | root | parent | next [-]

I think that's playing with semantics. Most codebases have old parts, the old part of texlive just has a name. It's still being actively maintained and would be a lot less usable if it wasn't, there's just an imaginary line between the texlive part and the tex part.

atomicson on Feb 5, 2021 | root | parent | prev | next [-]

Old version? You have to understand that some applications need innovation based on new ideas. TeX is only popular in the academic and publishing world, not like a web browser. Can we write a complex equation in TeX as easy as we write in popular word applications like MS Word?

btilly on Feb 5, 2021 | root | parent | next [-]

Can we write a complex equation in TeX as easy as we write in popular word applications like MS Word?

Thank you for the most ludicrous comment that I've seen today. The popularity of

TeX in academia is exactly because writing complex equations in popular word applications is painfully hard, and the typesetting is poor. By contrast writing them in TeX is easy and the typesetting defaults to excellent.

Talk to anyone who has to actually write many such equations. They will verify that it is not a question of "as easy". It is massively easier and better in TeX. Which is why academics working in math, physics and computer science overwhelmingly choose TeX.

jimmyvalmer on Feb 5, 2021 | root | parent | next [-]

> It is massively easier and better in TeX.

Obviously depends on where along the learning curve, and preformed command line literacy, which on a venue like HN is always assumed to be native.

As a hypothetical, consider a Rip Van Winkle situation in which a mathematician wakes from a coma he's been in since the 1970s. Now force him to typeset one of his monographs. He'll do it in MS Word.

btilly on Feb 5, 2021 | root | parent | next [-]

Obviously depends on where along the learning curve, and preformed command line literacy, which on a venue like HN is always assumed to be native.

Actually not.

As a hypothetical, consider a Rip Van Winkle situation in which a mathematician wakes from a coma he's been in since the 1970s. Now force him to typeset one of his monographs. He'll do it in MS Word.

I have personal experience pertaining to this.

Before I was a programmer, I was a graduate student in mathematics. I wound up in the early 90s having never used Word or TeX and in a position where I needed to type up a paper. I began with Word, and before long I complained about how hard it was. A fellow grad student said I should learn how to do it in TeX.

It was literally faster, *on the very first paper that I tried to type*, to learn TeX and then type my paper in TeX than it was to try to do it in Word. The visual result was also massively better with TeX. Typesetting math formulas in Word is simply that bad.

I've tried to typeset some simple mathematics in Word since. The experience has not materially improved when it comes to typing real mathematics.

Based on this personal experience, I am quite confident that in the Rip Van Winkle situation that you describe, the mathematician will wind up doing it in TeX. And do it the same way that I did. Try Word

because that seems easier. Ask a fellow mathematician when that proves to be a terrible experience. Be pointed at TeX and given a few tips. Discover that it is easier.

jimmyvalmer on Feb 5, 2021 | root | parent | next [-]

Okay, you win. Also: our individual opinions become vanishingly insignificant relative to the aggregate opinion of the market, one that continues to pay for MS Equation Editor.

btilly on Feb 5, 2021 | root | parent | next [-]

Also: our individual opinions become vanishingly insignificant relative to the aggregate opinion of the market, one that continues to pay for MS Equation Editor.

Actually they don't.

People preferred paying for products like <https://www.dessci.com/en/products/mathtype/> which allowed people to type TeX into Microsoft documents than they did Equation Editor. Therefore Microsoft gave up on Equation Editor. They then created an XML-based markup language for math, and MathBuilder around that. Which they then put a TeX translation layer into so that you can type simple TeX in Word, Outlook, and so on, then get a math equation out.

Sadly for Microsoft, they didn't actually remove Equation Editor. I say sadly because they eventually had to. Per <https://securityboulevard.com/2018/01/microsoft-kills-old-of...> it was found to have a serious security hole, and removing it was easier than fixing it.

Incidentally, despite having both TeX and MathML available to look at, Microsoft failed to turn out something as good as TeX for serious use. As a result most journals will not accept documents produced using Math Builder.

So the aggregate opinion of the market is in. TeX was better than MS Equation Editor. (Which is why TeX outlived MS Equation Editor.)

jimmyvalmer on Feb 5, 2021 | root | parent | next [-]

Ah, I see the "Equation Editor" moniker is now defunct. I've long been in the LaTeX camp, so I've no idea what's transpired since 1990s when I last used a quasi-wysiwig entry called "Equation

Editor."

How you managed to conclude LaTeX entry is now more popular than whatever quasi-wysiwig method MS Word currently supports is, I suppose, market information I'm not privy to.

atomicson on Feb 6, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

I wrote one. My point is, we need a better way to write a complex equation as easy as we wrote some texts like in MS Word or LibreOffice. That's an innovation. At least some guys in Microsoft or LibreOffice guys already did. Hide the complexity. Maybe one day you could write a complex equation only by voice command. No one will touch TeX anymore.

tomc1985 on Feb 4, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

Even if the project is 'dead' it still runs, particularly on Windows.

I'm running the final release of Winamp as I type this. I organize my hard disk with the 1.0 of Spacemonger, which was free before it went paid. I edit audio files with Sound Forge 11 (up to 14 or so now) and before that I had a pirated copy of 6.0 that worked pretty well. I have a 'programs' folder full of stuff that runs without installation, some of which hasn't been touched in 5 or 10 years, and everything still runs when I try it.

Code is eternal.

1996 on Feb 4, 2021 | [root](#) | [parent](#) | [next](#) [-]

> Code is eternal.

Especially win32 apps, thanks both to windows and wine

> Even if the project is 'dead' it still runs, particularly on Windows.

For software I want to run long term, I always get the win32 version.

asciidient on Feb 4, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

You are my software twin. I'm using Winamp 2.95 this very moment, and Spacemonger 1.0 which still works fantastically. Both have slight crashes in rare edge cases now but perfectly usable day-to-day. You wouldn't perchance also be using an old version of UltraEdit before the heavy focus on subscriptions?

tomc1985 on Feb 4, 2021 | [root](#) | [parent](#) | [next](#) [-]

Hah, no I paid for Sublime Text way back when, and before that I was big on Notepad++. I rarely update either though :)

navanchauhan on Feb 4, 2021 | [root](#) | [parent](#) | [prev](#) | [next](#) [-]

AutoDock Vina[0] is one of the most popular molecular-docking softwares used (if we go by papers citing it). It was last updated on May 11, 2011.

I personally hold the opinion that if the project is not being maintained, giving away the source code allows someone else to pick it up.

[0] <http://vina.scripps.edu/download.html>

ghaff on Feb 4, 2021 | root | parent | prev | next [-]

I think both can be true.

A snapshot of source code can be useful as a basis for some other project. However, you're also correct that, if the source code is abandonware and you/others have no interest in maintaining it, it almost certainly becomes less useful over time and at some point just breaks.

hombre_fatal on Feb 4, 2021 | root | parent | prev | next [-]

Your youtube-dl example doesn't make sense because youtube-dl wasn't wiped from everyone's computer nor from package managers nor from the internet. So of course losing youtube-dl wasn't the concern.

The thing at threat was the thing the team was using to maintain it, so that's what people were concerned about.

You don't care more about your garage than your house just because you're crying about your garage when a tornado wrecks it but not your house.

bennysonething on Feb 4, 2021 | root | parent | prev | next [-]

Thanks for writing the article, see my comment else where in this thread. It's really helped me. I just been through a rough ride of a handover.

navanchauhan on Feb 4, 2021 | parent | prev | next [-]

Context: They are the maintainer of Krita[0], a really popular open-source painting program.

[0] <https://krita.org>

rytill on Feb 4, 2021 | parent | prev | next [-]

> You won't be able to buy something like this for any sum of money, so it's free.

I don't understand this. Why won't you be able to buy something like this? Do you mean you won't be able to sell it? I suppose if the first condition is true, the second condition follows.

tangjurine on Feb 5, 2021 | parent | prev | next [-]

I'm really not trying to be snarky, but Adobe has something similar and you can buy it, right?

bjornsing on Feb 4, 2021 | prev | next [-]

I agree with the conclusions, but the premise that "source code is worthless" is click-bait and misleading.

After having gone through a few M&A discussions with a high-tech software startup this is how I think about it: Nobody wants to buy just the source code. Talent / team that is used to work together and has proven itself has some value (i.e. "talent acquisition"), but valuation won't be very high. What's really valuable is *talent and code (with IPR rights) combined*.

But don't think you can put the source code on GitHub under an MIT license and it won't affect your M&A discussion... ;)

ajsharp on Feb 4, 2021 | prev | next [-]

"When we hired a new COO, who had mainly worked at bigger companies before, he was shocked to hear that all our code, communication infrastructure and internal systems were living in the cloud. He argued that we should move to on-premise solutions as soon as possible, partially out of fear of intellectual property theft, partially to appease investors with similar fears."

[shudder]

DavidAdams on Feb 4, 2021 | parent | next [-]

Yeah, my first thought after reading that anecdote was that the new COO should have been fired.

gamacodre on Feb 4, 2021 | root | parent | next [-]

And the CTO too, for caving to that argument.

quelsolaar on Feb 5, 2021 | root | parent | next [-]

A team that needs 6 months to get their code running on a computer on-premise, might also be replaceable. Maybe the lesson is that that team that produces source code without value.

Quarrelsome on Feb 4, 2021 | prev | next [-]

When I used to contract I used to work for a firm that would waste their time panicking about this but the reality was that their code was so bad that if it was released and their competitors could read it then that would probably help us because they'd be wasting their time. Much of it was very hard to read, work with and needed serious improvements.

We don't sell the source code, we sell the ability to update, maintain, test and build that code and that's institutional and staff knowledge.

nojvek on Feb 5, 2021 | prev | next [-]

I resonated very strongly with the Theory of problems. At my last place I was very productive. Revamping their deploy system, frontend infra, had my hands in a lot of things.

However my pay wasn't going up by much. I left for a new company and I feel like a noob having to re-learn things. It's proly going to take me an year to produce more value than I get paid.

Old company proly had to replace me and spend 200k*2 years to get same context.

Very few companies understand the essence behind retaining good people. Promoting can be wayyy cheaper than having to hire someone new.

It really pisses me off when this happens to female and minority engineers. They frequently get looked down upon and left out for well deserved promos.

If you care about diversity, retain and grow your existing team. It's cheaper and faster.

mynegation on Feb 4, 2021 | prev | next [-]

"One of the fears from management was that a big company like Google could take our code and build a competing service."

Google, or any other respectable company, would not touch source code without a permissive license with a ten foot pole, least of all if it was retrieved as a result of improper access.

deepsun on Feb 4, 2021 | parent | next [-]

Two more reasons Google doesn't touch others code:

1. They have monorepo with custom build infra, which makes integrating any third_party code pretty hard.
2. In google career culture, one gets promoted for development of something new, not integrating existing proved solutions.

tester34 on Feb 5, 2021 | root | parent | next [-]

>They have monorepo with custom build infra, which makes integrating any third_party code pretty hard.

Why would it be significantly harder than creating new project?

notacoward on Feb 5, 2021 | root | parent | next [-]

Different company, maybe different problem, but importing code at Facebook was similarly difficult. The build issues were the least of it. More significant by far were the all-but mandatory requirements to integrate with the in-house deployment infrastructure, service discovery, background-task scheduling, metrics, logs, alerts, data structure libraries, RPC framework, etc. Your project already implemented some of those internally, or used open-source alternatives? Too bad. You could keep the core logic, but practically everything about how it connected to the rest of the world would have to be rewritten. Often, it just wasn't worth it, and a new "FB native" service reimplementing the same functionality was easier. If you didn't do it yourself, some other group would constantly be threatening to do it for you. It's hard to focus on code when you continually have to justify your project's very existence.

deepsun on Feb 9, 2021 | root | parent | prev | next [-]

One problem I know -- different versions. They actually do have a small amount of third_party libs, but due to "diamond dependency" problem, they are accepting only one single version of that dependency. And updating that dependency is a huge undertaking (even for small security bugfixes), so those libs are usually outdated.

Kinda similar to OS kernel organization: monolith vs. microkernel.

pavel_lichin on Feb 4, 2021 | parent | prev | next [-]

Is Google also out of engineers or something? I would imagine that the important thing about building a competing service isn't understanding the specific implementation, but seeing the value in what the service provides.

mynegation on Feb 4, 2021 | root | parent | next [-]

True, but the premise of the article and my comment revolves around taking or seeing the code.

cloogshicer on Feb 4, 2021 | parent | prev | next [-]

Agreed, I think that argument back then was just bad.

jjeaff on Feb 4, 2021 | parent | prev | next [-]

What determines if a company is respected or not? Is Uber a respected company?

username90 on Feb 5, 2021 | root | parent | next [-]

You can ask engineers that worked there about their processes. Google have extensive processes both for using external code and for engineers accessing user data. I worked for a while on a system that had some user data, and every single access was logged and reviewed to ensure that each data access was in response to a user filing an issue and only accessed the data needed to fix the users issue. That is a standard process implemented in all of Google.

Companies who doesn't have engineers talking about processes are therefore not trustworthy. Doesn't matter if it is due to them being too small to not have ex engineers, or them threatening their engineers into silence, or their engineers simply not caring about processes and therefore not talking. Never trust companies where only upper management makes statements about their processes.

castwide on Feb 4, 2021 | prev | next [-]

The article makes a lot of good points, but I disagree with the notion that source code is worthless. The true test of the source's value isn't what would happen if it got stolen. It's what would happen if it disappeared altogether.

dehrmann on Feb 4, 2021 | prev | next [-]

> But even Panic says in their blog post that they were not too concerned with competitors using their code either, mainly because it would quickly become outdated

And the legality bit? Big companies who aren't Uber know this isn't worth it, but even with smaller companies, if someone internally notices, the company could be in a lot of legal trouble. I haven't worked anywhere that would even consider using a competitor's code.

npsimons on Feb 4, 2021 | prev | next [-]

That's not the post's headline; why did someone cherry pick this subheading? Is this simple clickbaiting?

cloogshicer on Feb 4, 2021 | parent | next [-]

I think the title was changed by a moderator, but it's actually quite accurate now.

asiachick on Feb 4, 2021 | prev | next [-]

I agreed with a lot of the points and that often source code might not be valuable but.... Much of it sounded like only thinking of a small team doing something no one else care about.

Conversely we have github and the millions of libraries it hosts in the form of source code.

We also have examples of WebKit based on KHTML, Blink based on that. Electron/Edge/Brave/Vivaldi based on Chromium. There is no one programmer who has the entirety of Chromium in their head and I'm pretty confident most teams would not find it easier to write their own browser from scratch. Chromium or WebKit are embedded in many places. PS5 and Oculus Quest are two off the top of my head.

cloogshicer on Feb 4, 2021 | parent | next [-]

I don't think that Naur was arguing that it has to be a single developer who knows everything. It could also be multiple developers, where each one of them has loaded a sub-problem of a bigger problem into their head.

Usually in OSS projects you have a handful or so of contributors doing the bulk of the work, right? Imagine them all going away at once for one of those projects you mentioned. I feel the project would be in pretty bad shape.

Vinnl on Feb 4, 2021 | [prev](#) | [next](#) [-]

The relative worthlessness of source code can easily be seen when the situation is flipped around. When you're told you can look at e.g. a competitor's source code, some open source code somewhere, or a prototype that a colleague already built during a proof-of-concept, that will almost never make you go much faster.

Meanwhile, I think organisations underestimate the cost of attempting to keep source code secret. It adds friction to almost every step of the development process, compared to not worrying about whether your build logs or repositories are publicly accessible, even if they're not open source.

seanmcdirmid on Feb 4, 2021 | [prev](#) | [next](#) [-]

You just need great archeologists to dig some worth out of source code whose authors have long past. At least, that's the premise behind Vernor Vinge's programmer archaeologists as portrayed in the fiction *A Deepness in the Sky*. Dystopian views of how code will evolve in the future are useful to informing on these kinds of things.

notacoward on Feb 5, 2021 | [prev](#) | [next](#) [-]

Had a very similar moment at one of the startups I worked for, which also served as my introduction to a new CEO.

Shortly after NewCEO was hired, we had an all-hands introduction meeting. "All hands" at this point meant about 50 people, so our largest conference room was packed. I ended up sitting on the floor by the big whiteboard, opposite NewCEO. He started talking about how bigger companies were (he believed) actively trying to reverse-engineer our product. Without exactly meaning to, I said "good luck" loudly enough to get his attention, so he looked at me expectantly.

"We're having enough trouble engineering it forwards."

A few people laughed. NewCEO kind of glared. I guess I should mention that I was the product architect BTW. We never particularly got along, but he was neither the first nor the last CEO I outlasted during my career so meh.

Getting back to the point, *maybe* a competitor who had access to our code might, with great difficulty, learn something about the problem domain we were exploring (continuous data protection). More likely they would have been misled by all the remnants of wrong turns we'd taken during the exploration process. They'd literally be better off without it. Our entire business was a gamble that we could get to market before bigger competitors woke up to the opportunity and threw more bodies/dollars at it. As it turns out, that's exactly what did happen. We lost the bet. Other people having our code would not have changed that outcome one bit.

dwohnmok on Feb 4, 2021 | [prev](#) | [next](#) [-]

This is startup-centric and depends heavily on the velocity of your project. For intrinsically difficult problems that evolve slowly or not at all (the most extreme example of this would be e.g. unsolved CS problems), source code is gold and its theft represents a huge blow to any competitive advantage a product might have had. Note that this is different from saying the source code itself doesn't evolve.

For problems whose difficulty relies rather on business concerns and which evolve rapidly then (the theft of) source code is nearly worthless.

daotoad on Feb 4, 2021 | [prev](#) | [next](#) [-]

This fits nicely with some ideas I've been working on about "Tribal Knowledge". It's going to take some thought to integrate fully. I am primarily looking at the work of Ong on orality and the distinctions he draws between oral and literal cultures.

It's not uncommon to hear the phrase used as a pejorative, with the implicit assumption that oral culture is inferior to literate culture and that tribalism == primitivism == bad and undesirable.

But I've come to believe that there are always going to be things that cannot or should not be represented in written documentation. Documentation is an artifact that must be maintained. Individual professionals and organizations spend a huge amount of energy producing, managing, and maintaining written documentation.

But how much time have we spent on oral transmission of ideas? Almost none. And that's because we dismiss it as an approach. It's seen as less than or primitive, or at best something to discourage.

But if you accept that there must always be some oral information, then ignoring the issues of how we communicate, update, and persist that orally transmitted information seems foolish. There are cultures that have more recently made the transition from being primarily oral to literal that have not forgotten the skills needed to manage this class of knowledge.

I wonder what practices we can learn from them that we can use to improve our ability to share and communicate more effectively.

gmfawcett on Feb 5, 2021 | parent | next [-]

I don't know Ong's work at all. But I wonder where "lector" culture fits? If someone is reading or reciting to you, is that oral culture? or is dialogue a necessary component?

Really I'm wondering where the divide is between, "you need to talk with me to learn this", and, "you can subscribe to my channel to learn this."

krab on Feb 5, 2021 | root | parent | next [-]

I think a good lecture is always followed by a discussion. So the oral culture may be more inviting to the dialogue, even if it specifically doesn't require it.

Vice versa, we're now contributing to the literal culture by discussion and people have been writing replies to texts forever. However, the ratio of information read to the amount of information discussed is different. It's harder to find people willing to discuss in writing a random piece of text you just read as opposed to discussing verbally what the lector just said.

cloogshicer on Feb 4, 2021 | parent | prev | next [-]

Thank you for your comment and input from this direction, super interesting!

Reminded me a bit of Jon Blow's excellent talk here: <https://www.youtube.com/watch?v=pW-SOdj4Kkk>

Do you have any resources you can recommend on this topic?

pasquinelli on Feb 4, 2021 | parent | prev | next [-]

would the oral transmission of information be what's called in other industries training?

StillBored on Feb 4, 2021 | prev | next [-]

I think the author is seriously confused over the value of some crap his startup created in a couple man years vs actual products with market share, long term maintenance and thousands+ of man years of engineering/testing/documentation time. He even admits that they were rewriting large parts of it on a regular basis. That by itself indicates much of the code actually had little value, if the engineers themselves were throwing it away.

It would be really hard to convince MS/Google/apple/etc that their primary products source code doesn't have any value.

jazzyjackson on Feb 4, 2021 | parent | next [-]

I think the author makes a great argument, and would add network effects to reasons why code is worthless. Let's say I build an

exact clone of Facebook, do I take away any value from Facebook.com? How about Office 365, I offer an exact copy of their cloud, except the sync doesn't work because OneDrive is built into Windows and won't authorize with my clone.

I guess before everything sync'd and auth'd to the cloud I could pirate Photoshop, but as the author points out, cracked software has always been a different landscape that carries malware more often than not and has no stability or feature updates, so why worry, it's no real competition if you're actually innovating your product, not to mention tech support!

As for throwing out code, I'd have to do some digging but there's a talk, maybe Dan Geer, outlining that every 1000 lines of code you write there's a certain number of security vulnerabilities, and you'll never find them all -- and the longer the code stays the same the longer those vulnerabilities are able to be prodded and discovered. So say you have an adversary with access to your source code, they are trying to figure out the "weird machine" of all the bugs in your code. The best way to foil this adversary is to keep changing the way your software works, always switching out one set of undetected bugs for another. Again, having a development team that understands *how to change the code* is infinitely more valuable than having access to the repo.

StillBored on Feb 5, 2021 | root | parent | next [-]

(moving the security bits up because I find it dangerous)

And the security argument is a strange one. If you never let the code "mature" then your defect count remains high. Which means there are likely exploits that can be quickly found with simple automated tools, vs being hardened enough that it actually takes real effort to find the ever more obscure cases. Which is why when you look at windows, a lot of the exploits recently are because they churned pieces of the OS that were decades old. And the "unsupported" versions of the OS weren't vulnerable. Similarly, the product I was working on a few years back dodged heartbleed for the same reason. We were on a fairly old version of SSL only being patched with security updates. So, when the exploit finally became public we didn't have anything to worry about. Our version of SSL simply wasn't affected.

It's very dangerous to think that the most secure version of a product is the one that isn't battle tested because it's being churned. That is just a reformation of the security through obscurity argument and assumes there aren't blackhats more than happy to hack a product and keep quiet about an exploit for years. Combined with the fact that now you're hoping to randomly close these exploits through code churn just screams of a naive development model.

(comment on network effects)

I've rarely heard anyone mention any of the recent web based "innovation" as a reason to use photoshop over gimp, or even older versions of photoshop. OTOH, when I heard these discussions in the past, there were real hard reasons people didn't use gimp (color profiles?), libreoffice (document compatibility), etc. So the "innovation" needs to be something the end user finds useful, not just pretty buttons, or software subscription models.

It's obviously not enough to just appear to be a clone, there have to be real reasons to consider an alternative to overcome the network effects. When that happens you can bet people start choosing the "clone", which does in fact devalue the original offering. If a legitimate facebook, O365, etc competitor shows up you can bet people will start to switch even with the network effects of those two products. In the case of photoshop, from what I've heard a lot of people have been looking at Affinity's product. Which points to gimp still not being a proper alternative.

This isn't just software, it's everything. Everyone keeps buying x86, until the day it turns out there is a cheaper/faster arm laptop. And it might not even be a change in the products themselves, the US automakers lost out in the 1970's because the market changed and they weren't as well positioned for it.

cloogshicer on Feb 4, 2021 | parent | prev | next [-]

Large companies also do occasional complete re-writes of legacy products. It's not as common (because of course the larger the project, the more expensive it is), but it still happens.

I would also argue that rewriting your code somewhat frequently is part of good engineering. As you discover more of the problem, old code needs to be discarded. Usually what happens is that you start solving problem A, then realize a new need to solve problem B. But really it would've been much better to solve a combination of the two, problem C, which requires an entirely different approach than dumping new code for B onto the old codebase for A.

It's actually part of why I wrote the article, because I do honestly believe it is a common misconception that rewrites are more expensive than modifications (although I'm sure that's true in some cases).

Also, I gotta say, while I appreciate that you took time to comment and chime in with the discussion, the way you worded it was quite rude and a bit hurtful.

dgb23 on Feb 5, 2021 | root | parent | next [-]

> the way you worded it was quite rude and a bit hurtful

You probably just hit some nerve of the person you responded to. Granted, your article was exaggerating about the worthlessness of source code, you made a very good and interesting point grounded in real experience.

On top of that you did something that I think is very important, commendable and interesting: Looking at the history of software engineering and programming. There is a wealth of knowledge and insights at our fingertips and as a culture we're not paying enough attention to history.

And the message is very sound if not taken to the extreme. Source code quality matters and is worth investing in, but people ultimately matter more. It's an important message that needs to be heard again and again.

horsawlarway on Feb 4, 2021 | parent | prev | next [-]

I disagree with you... sort of.

I think there's some truth that code has value, and there is some risk that making it available can cut into profits (See: Redis, Mongo, etc, changing course after open offerings became available on AWS/Azure/GCP).

But I think the much larger truth is that *most* of the value provided by companies (and most of what they charge for) is not "lines of excellent code" but rather the operating expertise of keeping a complicated system stable and available.

For example - All of the companies you listed *do* have widely available, open source offerings (Android, VS-Code, Mono, Swift, Webkit, etc).

The value wasn't in the code, the value was in the ecosystem around it.

I think this is true in more cases than folks expect. The Windows source code was leaked, but I don't see any companies scrambling to compete with MS by building on that code.

I think even if most of Google's repo was made public - the valuable part was the team that supports the infrastructure behind it, not the lines of code themselves (or at least, they make up a smaller portion of the value)

tshaddox on Feb 4, 2021 | root | parent | next [-]

> I think there's some truth that code has value, and there is some risk that making it available can cut into profits (See: Redis, Mongo, etc, changing course after open offerings became available on AWS/Azure/GCP).

An example like that is only valid if you argue that there was a reasonable chance that the company could have 1) developed a comparable closed source version of the product and 2) somehow prevented a competitive open source version of that product from existing and being used by competitors.

StillBored on Feb 4, 2021 | root | parent | prev | next [-]

Well I don't disagree with that either. Only that I must point out that libre washing a company with an open source product here/there doesn't really count. Sure android is open, but google isn't using it to make money directly, instead it feeds into the closed source ad/marketplace offerings.

If they opened that code, or apple opened up the entire iOS stack its quite likely they would have competitors that as you point out lowered the value of their primary offerings.

A google with a half dozen competent ad/search companies would look very different than the one that can afford to give away a large part of their product portfolio.

So, there is value in operating a "service" buisness, but there is even more value in operating a service business that has high barriers to entry. One way to erect those barriers is with hundreds of millions of dollars in engineering time spent on "source code" be that the code actually doing the searches/etc or the code being used to manage the clusters its running on.

TheNewAndy on Feb 5, 2021 | parent | prev | next [-]

Which doesn't really explain why when most of the source code to Windows XP leaked, it barely moved the needle on anything.

cloogshicer on Feb 4, 2021 | prev | next [-]

The referenced article by Peter Naur (Turing Award winner, invented Backus-Naur-Form) is well worth a read.

ivan_ah on Feb 4, 2021 | parent | next [-]

Here is a link to a PDF version of this paper:

[Programming as Theory Building -- Naur 1985] <https://demo.cygale.net/naur1985programming.pdf>

p4bl0 on Feb 5, 2021 | root | parent | next [-]

Haha I wonder how did you came across this link?

This demo website might disappear or at least the pdf might be removed from it. The stable version of this PDF is on my web page here: <https://pablo.rauzy.name/dev/naur1985programming.pdf>

EDIT: *to the mods*: maybe the link in the parent comment could be updated and my comment deleted? (I am the maintainer of cygale.net — the demo website can be edited by virtually anyone... my personal webpage is much more trustable).

ivan_ah on Feb 5, 2021 | root | parent | next [-]

I found the link via Google Scholar search = <https://scholar.google.com/scholar?q=Programming+as+Theory+B...>

I was looking for a version of the paper with machine-selectable text (because the PDF linked in the OP is a scan with no OCR).

p4bl0 on Feb 5, 2021 | root | parent | next [-]

Okay thanks. I will put a permanent redirect to inform GScholar's index of the actual stable URL of the document. Text selection and accessibility is indeed why I took the time to build this proper PDF as I explained here before seeing your comment: <https://news.ycombinator.com/item?id=26035175>

nitwit005 on Feb 4, 2021 | prev | next [-]

The problem with saying it's about developers building up theory is that we all tend to forget quite a bit. Code tends to contain a ton of fixes, versioning details, or algorithmic tricks no one remembers.

I've certainly rebuilt something from scratch only to immediately stub my toe.

jariel on Feb 4, 2021 | prev | next [-]

Working code (beyond super basic CRUD) that maintains a companies revenues is 'priceless' not 'worthless'.

It's 'worthless' to outside parties, but that's completely besides the point.

I worked at a Networking startup that made it's own ASICs and was sold for billions on the legit premise of 'working silicon'. The plans were actually stolen by a contractor who walked out, and while bad, were a little besides the point because they can't reasonably be used by others.

It's like saying 'GM's factory is worthless'. Well maybe on the free market, but as an operating entity it's worth a lot.

And yes, 'autonomy' is a nice thing for senior devs, but it's also hard to do in practice.

YeGoblynQueenne on Feb 5, 2021 | prev | next [-]

Could we please change the title back to its original? There is no obvious reason for changing it on the HN post.

The title of the article is *How to hire senior developers: Give them more autonomy* and it's not about Peter Naur's view of programming, it's about the author's view on how to hire senior developers.

The article has a few select quotes from Peter Naur's article "Programming as theory building" but the quotes are mainly used to advance the article author's opinions on hiring senior developers, rather than Peter Naur's idea of what programming is. Peter Naur does not seem to be related to this article or its author at all.

mywittynname on Feb 4, 2021 | prev | next [-]

I feel like the Unix philosophy of very small programs that do one thing, and do it well, is an application of the concepts in this article.

> One of Naur's main conclusions is that making changes to an existing program (to accommodate changing requirements) is often more costly than writing new code from scratch, at least if done by people from a different team. This is because there are intangible aspects of the model/theory in the programmer's heads, which can't be expressed in code and documentation:

Small, single purpose applications require less maintenance (which is expensive), but proper design allows them to be combined to create more complicated programs.

daotoad on Feb 4, 2021 | parent | next [-]

But to compose something large you begin to need hundreds of these small tools interacting.

Do you build slightly larger tools that integrate the small tools? Even these larger tools my need to be composed into another level of abstraction to build the final system.

This is essentially the same architecture that you would build with a layered set of abstractions and interfaces.

It's really just a matter of how the different bits communicate that is different.

username90 on Feb 5, 2021 | prev | next [-]

> Your source code is worthless

This disregards the effect of having users. Once you have built something and lots of people start using it then the code for the library becomes the main source of truth for what that library should do, since any changes to its behaviour is likely to break someone's workflow.

This is why making "clean rewrites" is so hard, since reimplementing all the peculiar behaviour of the previous implementation is much much harder than implementing new peculiar behaviour for a new library with less well defined requirements.

Source code isn't particularly useful for competitors, no, but it is extremely valuable for your company.

p4bl0 on Feb 5, 2021 | prev | next [-]

The article from Peter Naur is a must-read. And what it says is absolutely not limited to programming. It is a politically deep and important paper that applies to virtually every human activity that adds value.

I recommend reading this paper to all my computer science students.

I took the liberty to make a better formatted and easier to read PDF of it a while ago: <https://pablo.rauzy.name/dev/naur1985programming.pdf>

zwieback on Feb 4, 2021 | prev | next [-]

Provocative, exaggerated title but a great article.

I would argue that many bodies of source code are worth something even if they were handed to me without any additional info. Let's say I'm working with a new microcontroller and someone gives me the source to a TCP/IP stack for that new micro. From the data sheet of the micro I could make sense of the lower levels of the stack and TCP/IP is so well defined that just the value of someone making it work on the particular hardware, which is mostly busywork, has huge value.

dbrueck on Feb 4, 2021 | prev | next [-]

The article is great, though a more accurate (but admittedly less thrilling) title would have been "Poorly Documented Source Code Is Virtually Worthless".

Good source code includes level of intent comments, it explains the reasoning behind decisions (especially the tradeoffs that were made) and warns future readers of potential pitfalls.

In both a professional setting as well as when wanting to contribute to a favorite OSS project, few things are more frustrating than jumping into the code only to realize it's missing this type of info.

jimmyvalmer on Feb 4, 2021 | parent | next [-]

> "Poorly Documented Source Code Is Virtually Worthless".

And richly documented code is non-virtually worthless.

If you don't believe me, try parsing one of these so-called "literate programs." Any 10-year vet will far more quickly grok a piece of code when its textual flow isn't disrupted by verbiage that only makes sense to its author.

jayd16 on Feb 4, 2021 | prev | next [-]

"Kill your darlings" is probably a better aphorism.

Obviously source code is useful on its face, and its also the truest documentation of current requirements. Worthless is an overstatement.

StillBored on Feb 4, 2021 | prev | next [-]

Thinking about this more, I wonder if you can predict whether a software project is "failing" by the amount of code that is getting deleted early on.

I've been part of a half dozen or so successful medium->large scale projects, and we never really deleted large parts of the code base. The small teams bootstrapping them had a pretty clear goals of where they were going sufficiently to build a reasonable scaffold early on to get somewhat working prototypes. Then as the project took shape things were fleshed or refactored far more than they were thrown away.

OTOH, projects i've seen fail frequently had core pieces rewritten on a somewhat regular basis.

Which if you think about it in the sense that every time a chunk of code is wholesale replaced, that is X man hours of effort flushed down the toilet. Few startups can afford engineers to be working on things which don't go towards the bottom line.

AKA if you have 6 engineers, and on average are replacing 25% of the code base a year, that is the equivalent of just throwing away 25+% of your runway every year. With the team communications overhead/etc I could totally see that it might be possible to never actually make forward progress even though the amount of code being thrown away isn't >50%

fwip on Feb 4, 2021 | parent | next [-]

The amount of code written isn't static, and doesn't map neatly to "amount of developer effort."

If everything you write has to be "perfect" the first time around, you'll write a fraction as much as if you're willing to build 3 things (either now, or in the future), and take whichever works out the best. It might take more time, or it might take less time.

paxys on Feb 4, 2021 | prev | next [-]

I have lost count of the number of startup founders I come across who have an unsuccessful product and are looking for an exit and go "we have all this amazing code. It has to be worth at least \$X". No, all of your code is worth exactly \$0 to any company looking to acquire you. They either want your product (read: users) as a whole or just the talent behind it. In 99% of such acquisitions your entire codebase will be thrown out.

ncmncm on Feb 4, 2021 | parent | next [-]

I was at a startup that ran for two years and wrote a lot of code.

At the liquidation, the entire body of code from the two years was auctioned for \$1372 to one of the founders. Chances are, if he had not bid, it would have been abandoned entirely. I doubt he looked at it, after.

aeldidi on Feb 4, 2021 | prev | next [-]

Rather than "Source Code is Worthless", I find it makes more sense to think of the lesson learned here as "Source Code is Only Worth Something to the Public if it's Well Documented and Stable", which happens to be true more often in practice.

For example, someone I used to know would always say that while the Linux kernel as a product (a free and open source operating system) is priceless, the source itself is worthless unless you have an understanding of its internals, and I think that this is generally true for codebases of similar size and complexity, although I'd argue Linux specifically doesn't fall into this area only because of the fact that drivers make up most of the code in it.

Stability is especially important when considering long term worth of code. When youtube-dl was removed from GitHub not too long ago, people were worried that the project would stop being updated and maintained, despite the fact that mirrors of the source code were everywhere. In that case, because the project needs constant updating and maintaining to keep up with websites like YouTube who have no incentive to continue to keep a stable API for downloading videos.

d4v3 on Feb 4, 2021 | prev | next [-]

I agree with the conclusions drawn in the article, but not the premise. Source code is definitely not worthless. Two examples:

- 1) using source code to find security exploits
- 2) competitive machine learning algorithms

In the second case, imagine if a competitor had access to algorithms used in marketing for bidding on ads. You know what features it uses, what default values it uses, how it works, etc. You could do some damage with that knowledge

enobrev on Feb 4, 2021 | parent | next [-]

It seems that you not only agree, but argue further that source code is a liability.

d4v3 on Feb 4, 2021 | root | parent | next [-]

Hah, nice try, but no. Arguing that something is a liability if it fell into the wrong hands does not mean it is worthless or worth less than zero. It simply means it is worth protecting.

jhallenworld on Feb 4, 2021 | prev | next [-]

Source code is most valuable to the person who wrote it / knows it intimately. If I can use my code on different jobs it will definitely save me time. For this reason I try to make any library code MIT licensed. This allows me to use it for different clients without them complaining. Our contracts are set up to allow this.

Often the customer's proprietary code is a small fraction of all the code.

kisssgyorgy on Feb 4, 2021 | prev | next [-]

This is because there are intangible aspects of the model/theory in the programmer's heads, which can't be expressed in code and documentation

But it can be expressed in commit messages and "why" comments! This is very important a lot of developers don't understand and want to just "get on with their life" after writing the commit message "fix".

If used properly, version control can be the repository for all ideas stored historically and can be replayed. With well-written commit messages and well-formatted commits, whole ideas can be followed and mistakes can be recognized years after the code has been written. I have seen multiple times when we recognized "oh, they wanted to do this or that, but they forgot to change this", so we could fix it, because we understood the original idea someone had 10 years ago!

I believe, THE PRODUCT of Software Engineers work is not source code, but COMMIT MESSAGES!

lumost on Feb 4, 2021 | parent | next [-]

This works to some extent, however it's pretty rare that engineers go more than 1 commit deep into the git blame.

kissgyorgy on Feb 4, 2021 | root | parent | next [-]

No it's not at all. I do it daily. You have to learn the tools you use every day!

cloogshicer on Feb 4, 2021 | parent | prev | next [-]

While I agree that good commit messages help, I think a lot of this info can't be reasonably conveyed through them.

While the analogy might not be perfect, it's kind of like trying to teach someone how to play the piano by asking them to read a book.

groby_b on Feb 4, 2021 | prev | next [-]

If you accept the idea that programming is theory building, it's not that difficult to see when source code isn't worthless - when it's not the slow slog of building a theory entirely of well known facts, but when it contains leaps of insight.

There is also value in knowing which particular theories you're fleshing out vs. ignoring.

pan69 on Feb 4, 2021 | prev | next [-]

Source code isn't worthless, it's just not as important as the developers who wrote it because those developers can write a next, much better version, of the same software with the domain knowledge they have acquired writing the previous version(s). In practice this means that source code is disposable.

Ericson2314 on Feb 4, 2021 | prev | next [-]

"source code being worthless" is actually a huge red flag that something is deeply wrong with the way our industry works. There is too much in-house knowledge that isn't being codified. Software production shouldn't be like e.g. Jet engine production, where it's all nontransferable lore.

I posit if everyone was automatically build able and deployable (say with Nix) across the board. Source code would have more value.

Now, it still would be true that less source code is better than more source code, so line by line / marginally source code is a liability. But reproducible functionality would be valuable.

The remaining worthless might have to do with much especially b2b proprietary software being utter crap, and all the value being in the sales relationship and none in product/service being traded.

paxys on Feb 4, 2021 | parent | next [-]

You could do all that and the code would still not have any intrinsic worth, because if a company gets their hands on it they don't automatically get a team of developers with domain expertise to maintain it going forward. That's where the real value is. Knowledge and experience can never be codified.

StillBored on Feb 4, 2021 | root | parent | next [-]

So your saying it would take someone the same amount of time to say recreate an OS like windows, without the source code as it would to become proficient in building and maintaining it?

I don't buy that, if I wanted to create a \$HIGH DOLLAR application, and it was possible to get away with stealing the source code i'm sure it would happen more frequently.

It's not that the code for windows/etc is worthless, it's that anyone attempting to make any money selling a clone that was obviously based on it would be put out of business.

If OTOH, they sold a license to resell products based on the XP64 code base you can bet there would be LTS/embedded/etc type companies pooping up selling their own versions.

So while the effort to get a functioning build system and learn the code base is considerable, its a lot easier than reinventing the wheel. Hence why the code `_DOES_` have value.

Ericson2314 on Feb 5, 2021 | root | parent | prev | next [-]

I usually read "domain expertise" as "communications failure"

The purpose of the code is as much to communicate with other humans as it is to communicate with computers.

ErikAugust on Feb 4, 2021 | parent | prev | next [-]

I think it's incentives, partially.

Programmers say their job is to put themselves out of their jobs. People don't want to be out of their jobs, really. And we know that doesn't happen. If anything, it locks them into their job, as "subject matter experts".

Also, most applications are built on a million layers of reusable abstractions. Even in the highest level of the code people are using frameworks and other libraries. A lot of source code is just business logic and glue code when you remove the stuff it is built upon, which tends to be much better (and many times, open and free).

Ericson2314 on Feb 4, 2021 | root | parent | next [-]

> Programmers say their job is to put themselves out of their jobs. People don't want to be out of their jobs, really. And we know that doesn't happen. If anything, it locks them into their job, as "subject matter experts".

I agree it is a fools errand to ask people to work against their incentives, but by https://en.wikipedia.org/wiki/Jevons_paradox I don't think there is actually disincentive given enough risk tolerance to let the new equilibrium emerge. Write better programs, and increasing the demand for programming.

I think the bigger problems is most programs are bad and alienated. Programming with everything nicely packaged and ready to be modified is like working with a clean workbench / shop. If you've never been in a clean one, cleaning seems like a chore, but if you have, working in a dirty one seems not only inefficient, but gross and undignified.

It's "take care of your tools, and your tools take care of you" vs being "lord of my garbage heap of tech debt". Any self-respecting craftsperson should choose the former.

NateEag on Feb 5, 2021 | root | parent | next [-]

> I think the bigger problems is most programs are bad and alienated. Programming with everything nicely packaged and ready to be modified is like working with a clean workbench / shop. If you've never been in a clean one, cleaning seems like a chore, but if you have, working in a dirty one seems not only inefficient, but gross and undignified.

You hit the nail on the head here.

I've worked a bunch of different jobs since college.

I have worked in exactly one clean, mature codebase.

The difference to all the others I've been in was like night and day.

I think seeing even "acceptably decent" code is incredibly rare in our industry.

To be fair, that's probably in large part because producing acceptably decent code is prohibitively expensive for any new product, and once you've gotten a new product off the ground and producing value, you've set the cultural norm that crappy code is what we make here.

TheJoYo on Feb 4, 2021 | prev | next [-]

> They would be much faster just building it themselves from scratch, than trying to understand the crummy code we had hacked together.

They might use your code as a reference, especially from any research components of the project.

You'd be surprised what automated analysis of gitlab repos bring to light.

gfody on Feb 4, 2021 | prev | next [-]

source code is excrement. an unfortunate side effect necessitated by the immaturity of our tools. someday we'll make software without it and life will be much better for the poo tenders.

IggleSniggle on Feb 4, 2021 | parent | next [-]

I disagree. It's the offloading of a decision space very carefully explored. Perhaps "language" and "syntax" can be generalized (ie "excrement"), but the ability to offload a decision space about how to react to a given scenario, without requiring someone to be actively thinking about the minutiae of the problem space, will always have value.

Whether that takes the form of a dependency graph in a software application or a set of assumptions (with their corresponding citations) in a scientific journal, the decision tree (inclusive of dependency graph) will always be essential.

gfody on Feb 4, 2021 | root | parent | next [-]

if source code takes the form of human language I would no longer call it source code - it still might be excrement but not necessarily so.

IggleSniggle on Feb 5, 2021 | root | parent | next [-]

I feel as though many problem spaces are already expressible in human language, but that "code" is just a more concise expression of the same thing.

The main issue with common language encoding is dialect (this happens in code, too, especially "common" languages like C++, Java, or JavaScript). That is to say, the assumptions you bring with you about what for example a "schedule" is affects all subsequent decisions based on it, but there are many possible interpretations for the semantics of such a thing.

It seems to me that most programming languages of today are *better* than "human language." They more concisely AND precisely express the decision space.

I assumed "better tooling" meant some deeper heuristic wherein you might expect an AI to interpret your meaning *based on your own enculturation*, accepting the high subjectivity of any request/definition and producing an output formed by these assumptions.

I would still call this "source code" however, in much the same way that legal precedent is the source code for the next legal decision.

blindm on Feb 4, 2021 | prev | next [-]

> Communicating the mapping of the code and the real world, as Naur describes, would have required us to document all these processes in meticulous detail. If we had tried to, I believe the result would've been multiple hundred pages describing the business processes

Or in other words: building in *business logic* is paramount and the *only* thing you should be doing if running a startup, otherwise these engineers are just dossing around with code that doesn't yield a profit.

shahrk on Feb 4, 2021 | prev | next [-]

Unless you are Waymo and the theft is by one of your employees.

sriku on Feb 5, 2021 | prev | next [-]

This seems to offer a criterion to decide the value of a business - if your business ceases to be one by publishing all your source code free to use, then you should look at other business models?

Is ElasticSearch (the company) then just a "bad business model"? Won't Google suffer if Amazon just waits for them to release their code and turns it into a managed service in competition? (Both companies are eminently capable here).

mcv on Feb 5, 2021 | prev | next [-]

I completely agree with this. I never find documentation very useful, and I'm fully aware I've got tons of theory in my head that I couldn't begin to write down. I should try harder because I won't stay on this project forever, and there are enough complex concept involved that aren't immediately obvious. But how do I write that down? And where do I start?

ajsharp on Feb 4, 2021 | prev | next [-]

I appreciate a good catchy headline as much as the next person, but it's a ridiculous claim that "source code is worthless". On some level maybe it's not worth as much as some might make it out to be, but it's not worthless. There's a ton of strategic insight to be derived from the source code, not to mention just a basic time-value of money level of value.

throwawaygh on Feb 4, 2021 | prev | next [-]

> *Especially for the lower-level components, a good amount of research had been involved. We worked closely with an academic institute that specialized in computer vision. In short, this project wasn't something that anybody could just easily recreate. So one of the assumptions we had implicitly made was that our source code was one of the company's major assets.*

Why not patent?

superjan on Feb 4, 2021 | prev | next [-]

This is easier to reason about if it's not about your own code. Consider this: would you swap your own codebase for your competitor's if you had the chance? For me, it's a resounding no. I bet it's the same from their perspective.

FrozenVoid on Feb 4, 2021 | prev | next [-]

Source code of a large monolithic projects(Products) lacks value due not being adaptable or modular, but it has worth that can be decomposed and refactored into other software(libraries/packages/headers/functions).

jimmyvalmer on Feb 4, 2021 | parent | next [-]

I'll speak for all the ten-year vets who know stripping a monolith for parts is often as time-consuming as writing from scratch, and always uglier. Software is not a car.

breck on Feb 4, 2021 | prev | next [-]

The more you design your program for the current world, the faster it will go out of date. Try to identify what will still be useful in 30 years, and get that part right.

iamwil on Feb 4, 2021 | prev | next [-]

Anyone read the post author's book, Hiring Engineers? Is it any good and worth the \$75?

cloogshicer on Feb 5, 2021 | parent | next [-]

Hey, author here. What's the situation that you're in? You (or anyone else reading this) can ping me at alexander [at] hiringengineersbook [dot] com, maybe I can help you figure out if the book is good for your situation or give some advice on the spot.

No worries, not trying to sell anything (other than the book), just always curious what challenges people are facing when hiring.

oaiey on Feb 4, 2021 | prev | next [-]

It is very often a developer perspective that the source code and the software product is the core delivery. Interestingly that is often not true. Customer care, project management, sales, marketing and so many roles contribute so much more to the value creation than often visible to a developer.

cloogshicer on Feb 4, 2021 | parent | next [-]

I completely agree, that's why I wrote in a footnote:

> However, I believe this statement is only true for software companies where the core technology is the main asset. For many software companies, the main value might not lie in their technology, but in other things, such as the network effects, relationships to customers, etc.

Pete-Codes on Feb 4, 2021 | prev | next [-]

Re your newsletter form, I'd take out the first name requirement.

cloogshicer on Feb 4, 2021 | parent | next [-]

Thanks for your suggestion! It's actually optional. I had a problem with spam signups and thought it might help a bit.

yudlejoza on Feb 4, 2021 | prev | next [-]

- There's no such thing as programming
- There's only research programming

TheJoYo on Feb 4, 2021 | prev | next [-]

I'm missing where autonomy comes into this.

Simplicitas on Feb 4, 2021 | prev | next [-]

Hyperboles will always derail any conversation

cccc4all on Feb 4, 2021 | prev | next [-]

Source code is worthless AFTER it's written.

It still has to be written first.

jimmyvalmer on Feb 4, 2021 | prev | next [-]

tldr: Code needs constant updating, and a coder who hadn't originally wrote the code will almost certainly make distasteful and often incorrect modifications. Ergo, code is worthless.

editorial: Most devs with >5 years experience know this. The digitization of content made movies, music, books, and software all but financially worthless. Split-second, error-free replication has rendered the IP of so-called "knowledge workers" far less valuable (movie studios, recording labels, newspapers, programmers).

dang on Feb 4, 2021 | prev [-]

That's an egregiously baity title. That's not cool here: <https://news.ycombinator.com/newsguidelines.html>

Edit: The submitted title was "Your source code is worthless". I've changed it now.

ontouchstart on Feb 6, 2021 | parent | next [-]

Dang, your title definitely made me curious enough to click the link, had a quick glimpse of the article and went to download the original 14 page PDF the author provided

<http://pages.cs.wisc.edu/~remzi/Naur.pdf>

This is the power of web and http that opens the world to any curious mind. Your change of title made a book marketing page to a gate to new ideas to explore and discover.

Also out of curiosity I traced the HN list of the submissions of the original link, i.e., the author who is promoting his book:

<https://news.ycombinator.com/submitted?id=cloogshicer>

You can see that your change increased the attention to his marketing effort by a factor of 2!

Unfortunately this attention would hardly help him reach his audience, who are on the hiring side of the market and have completely different constraints than the engineering minded people dominated in HN.

Interesting conversation though.

cloogshicer on Feb 14, 2021 | root | parent | next [-]

I mainly submitted this here for the discussion. I really enjoy reading people's comments and I'm very curious if my hypotheses are correct.

wnoise on Feb 4, 2021 | parent | prev | next [-]

And yet the current title ("Peter Naur's view of programming") now contains even less information about the content of the

article. It may be less "clickbaity" -- I certainly am less inclined to click on it, but it's also less useful.

dang on Feb 4, 2021 | root | parent | next [-]

I would say it contains much more information, in addition to being accurate.

If anyone can suggest a better (i.e. more accurate and neutral) title, we can change it again.

wnoise on Feb 5, 2021 | root | parent | next [-]

I'm not sure what you mean by neutrality here; I want the title of a piece to accurately reflect the thesis of the piece, whether or not the piece itself is biased.

But the current HN title is not a thesis at all. At best it's a characterization of the thesis of the piece. It's not a wrong characterization, but I don't see how it's useful. Certainly, he is indeed illustrating and referring back to Peter Naur's description of programming as theory-building, but this title doesn't tell me (a) what Peter Naur's view is, nor (b) why I should care, nor (c) whether this piece even agrees or disagrees with it.

The first subheading of "Your source code is worthless" is a running theme that connects everything. Yes, later on he calls to Naur's "Programming as Theory Building" article as an explanation for this observation, but the belief is more central than the explanation. It starts before the explanation and continues past the engagement with Naur.

I don't see how weakening this strong thesis by hiding it helps make things either more accurate or more neutral. Is it that it seems like an insult? Leave out "your", or change to "most" or "in general".

(In contrast, the article's title of "How to hire senior developers: Give them more autonomy" would be bad one. It is barely touched on, just thrown in at the end, as a conclusion on top of the meat of the piece.)

If I had to suggest something that covered what I consider the main points, and had more than the central theme: 'Source code alone is worthless as "Programming as Theory Building" suggests', but that's starting to attribute a stronger point of view to Naur than may be reasonable.

But the originally submitted title was perfectly okay.

cloogshicer on Feb 4, 2021 | parent | prev | next [-]

My apologies, dang. I thought it was borderline, but it captured the essence of what I was trying to say. Sadly it kind of derailed the discussion a bit, so I guess you're right.

Thanks for changing it instead of deleting it. How do you feel about "On the Value of Source Code"?

jimmyvalmer on Feb 4, 2021 | parent | prev [-]

It's baity because it's true.

dang on Feb 4, 2021 | root | parent [-]

That is not an adequate theory of baitiness.

[Guidelines](#) | [FAQ](#) | [Lists](#) | [API](#) | [Security](#) | [Legal](#) | [Apply to YC](#) | [Contact](#)

Search: